



University of Information Technology and Sciences

**Computer Graphics and Multimedia Lab
CSE 358**

Project Report for Triangular Jump

A fast-paced arcade-style game

Submitted By

**Sadia Jannat
0432320005101108**

**Fatema Akter Nishi
0432320005101121**

**Reyazul Islam
0432320005101146**

Submitted To

Any Chowdhury

Audity Ghosh

Lecturer

Lecturer

Department of CSE

Department of CSE

Table of Contents

Table of Contents.....	1
1. Introduction and Motivation.....	2
1.1 Introduction.....	2
1.2 Motivation.....	2
2. Dataset / Game Feature Details.....	2
2.1 Game Objects.....	2
2.2 Physics Parameters.....	3
2.3 Obstacle Parameters.....	3
2.4 Difficulty Scaling.....	3
2.5 Controls.....	3
2.6 Scoring.....	4
3. Methodology with Block Diagram.....	4
3.1 Rendering Pipeline.....	4
3.2 High-Level Architecture.....	5
3.3 Physics Integration (Per Frame).....	6
4. Pseudocode.....	6
4.1 Main Program Structure.....	6
4.2 AABB Collision Detection.....	9
5. Terminal Screenshots with Output.....	9
5.1 Build Output.....	9
5.2 Project File Tree.....	10
5.3 Window Title -Live Score Display.....	10
5.4 Game Visuals Description.....	10
6. Discussion and Future Work.....	10
6.1 Discussion.....	10
6.2 Limitations.....	11
6.3 Future Work.....	11
7. Conclusion.....	12

1. Introduction and Motivation

1.1 Introduction

Triangle Jump is this minimalist 2D arcade game made from scratch, using OpenGL 3.3 Core Profile and GLFW, yk. Players play as a yellow triangle placed on the left side of the screen. Cyan rectangular hurdles continuously pop into existence from the right, and they move left, sort of steadily but the speed keeps going up, a bit. The job is to jump over them, so players can stay alive, and also gather scores over time.

Everything is done inside one C++ file, main.cpp, kind of all in one place. It includes the OpenGL loader (GLAD) inline via `#include "glad.c"` and GLFW for window setup and input handling. Plus the whole deal: game logic, physics, shader code, and rendering. Basically the GPU pipeline is all wired up right there.

1.2 Motivation

This project motivation is three parts,

- **Learning Core OpenGL:** Instead of depending on higher level engines, building straight with raw OpenGL makes you touch the basics head on. Like, VAOs, VBOs, shader programs, uniforms, and the full rendering pipeline.
- **Simplicity as a Design Goal:** Modern game dev can turn into huge frameworks everywhere. This one intentionally reduces the whole thing to something very plain, two triangles and a rectangle, just to show that real time interactive graphics doesn't have to be complicated. Classic Endless
- **Runner Mechanics:** The endless runner style, the one made famous by stuff like Google's dinosaur game, is a known and fun gameplay loop. Making it from scratch in OpenGL becomes a great practice, since you have to blend physics simulation, collision detection, procedural spawning, and real time drawing, all together.

2. Dataset / Game Feature Details

Because it's a game project (not a data-driven application), this part is really about the game settings, the stuff that's in, and how the mechanics work, instead of classic dataset description.

2.1 Game Objects

Object	Shape	Color	Description
Player	Triangle	Yellow (1.0, 0.85, 0.2)	Controlled by the player , jump happens when Space is pressed
Obstacle	Rectangle (2 triangles)	Cyan (0.3, 0.85, 1.0)	Spawns from the right, then moves left

Object	Shape	Color	Description
Player (dead)	Triangle	Red (0.9, 0.25, 0.2)	On collision it turns red

2.2 Physics Parameters

Parameter	Value	Description
GRAVITY	-4.0 units/s ²	A downward acceleration gets applied every frame
JUMP_VEL	2.5 units/s	When jump there's an initial upward velocity
GROUND_Y	-0.65 (NDC)	There's a specific Y-coordinate where the player settles
PLAYER_X	-0.55 (NDC)	The player stays at a fixed horizontal position

2.3 Obstacle Parameters

Parameter	Initial Value	Description
speed	0.55 units/s	Obstacles slide horizontally at a certain movement speed
spawnInterval	1.8 seconds	Set time delay between obstacle spawns
MAX_OBS	4	Capacity for how many obstacles can be on screen
Obstacle width	0.07 (NDC)	Player and hit area use a half-width of ± 0.035
Obstacle height	0.30 (NDC)	Obstacle vertical size comes from GROUND_Y, from that baseline

2.4 Difficulty Scaling

The difficulty ramps up every 5 points: - obstacle speed increases by +0.08 units/s - spawn interval decreases by -0.15 seconds (but not below 1.0 second)

2.5 Controls

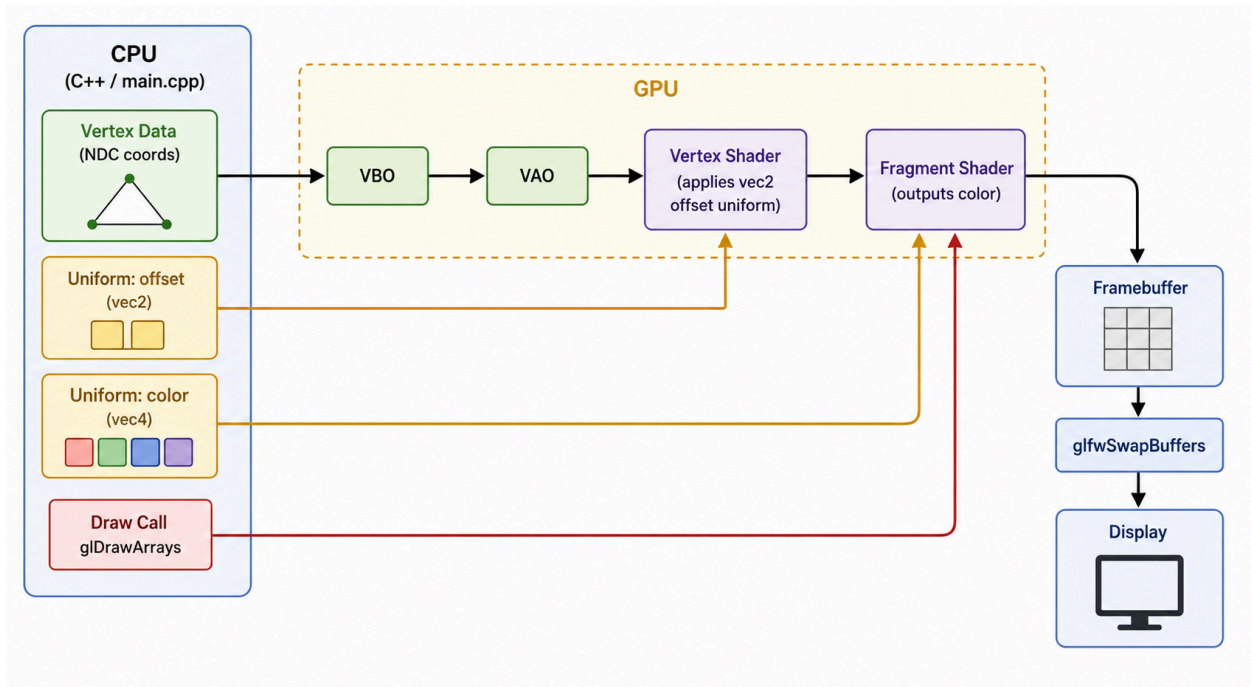
Key	Action
SPACE	Jump if playing Restart if crashed
ESCAPE	Quit window

2.6 Scoring

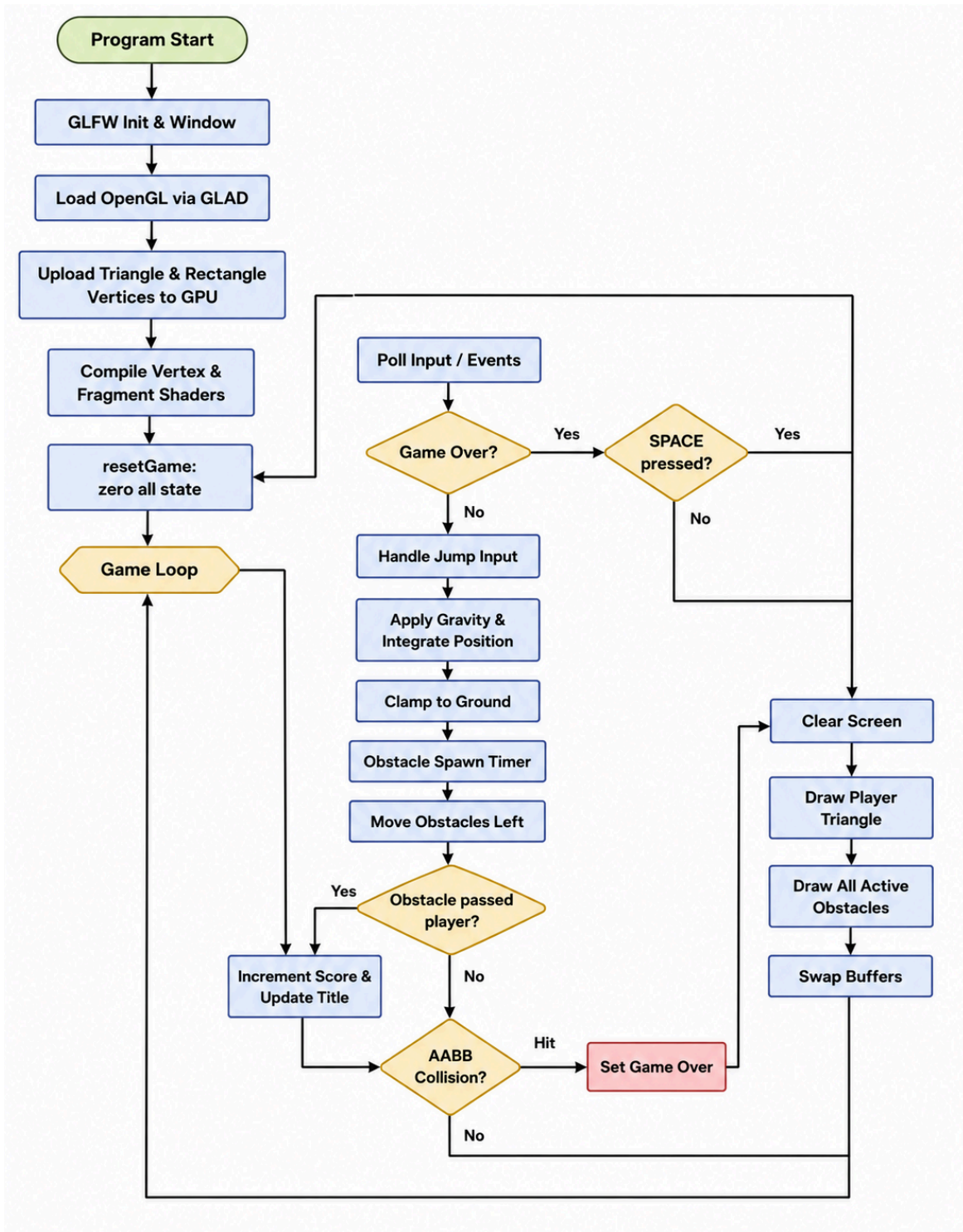
- Player earns +1 point each time an obstacle fully passes the player
- Score is shown live in the window title bar
- When the game ends, the final score is also displayed in the title bar

3. Methodology with Block Diagram

3.1 Rendering Pipeline



3.2 High-Level Architecture



3.3 Physics Integration (Per Frame)

```
velY += GRAVITY * dt // Euler style, constant acceleration thing

posY += velY * dt // update the height, basically

if posY <= 0 // hits the ground or near enough
    posY = 0, velY = 0 onGround = true, just clamp it
```

4. Pseudocode

4.1 Main Program Structure

PROGRAM TriangleJump

```
DEFINE shaders (vertex + fragment)
DEFINE constants: MAX_OBS, GROUND_Y , PLAYER_X

GLOBAL: posY , velY , onGround , gameOver , score
GLOBAL: speed, spawnTimer , spawnInterval
GLOBAL: obs[MAX_OBS]

FUNCTION resetGame():
    SET posY=0, velY=0,onGround=TRUE
    SET gameOver = FALSE, score=0
    SET speed = 0.55 , spawnTimer=0, spawnInterval=1.8
    FOR each obstacle : SET obstacle.active = FALSE

FUNCTION main():
    INIT GLFW create window 700x500
    LOAD OpenGL via GLAD

    UPLOAD triangle vertices to triVAO
    UPLOAD rectangle vertices to obsVAO
    COMPILE and LINK shaders → prog
    GET uniform locations, uOffset , uColor

    CALL resetGame()
```

```

prevTime = currentTime()

LOOP WHILE window not closed
    POLL events
    IF ESC pressed EXIT , return

    dt = currentTime() - prevTime
    prevTime = currentTime()

    IF gameOver
        IF SPACE pressed CALL resetGame()
    ELSE
        CALL handleInput dt
        CALL updateObstacles dt

    CALL render()
    SWAP buffers

FUNCTION handleInput(dt):
    IF SPACE just pressed , AND onGround
        SET velY = 2.5
        SET onGround = FALSE
    velY += -4.0 × dt
    posY += velY × dt
    IF posY ≤ 0.
        posY=0 , velY=0 , onGround=TRUE

FUNCTION updateObstacles(dt):
    spawnTimer += dt
    IF spawnTimer ≥ spawnInterval
        SPAWN new obstacle at x=1.2
        RESET spawnTimer

FOR each active obstacle:
    obstacle.x -= speed×dt

```



```

IF obstacle.x < PLAYER_X AND NOT scored :
    score += 1
    UPDATE window title
    IF score % 5 == 0:
        speed += 0.08
        spawnInterval -= 0.15

IF obstacle.x < -1.3:
    SET obstacle.active = FALSE

IF AABB(player, obstacle) overlaps , THEN
    SET gameOver=TRUE
    UPDATE window title with final score

```

4.2 AABB Collision Detection

```

FUNCTION checkCollision(player, obstacle) => BOOL :
    px0 = PLAYER_X - 0.06    // player left boundary
    px1 = PLAYER_X + 0.06    // player right boundary
    py0 = GROUND_Y + posY    // player lower edge
    py1 = py0 + 0.10         // player upper edge

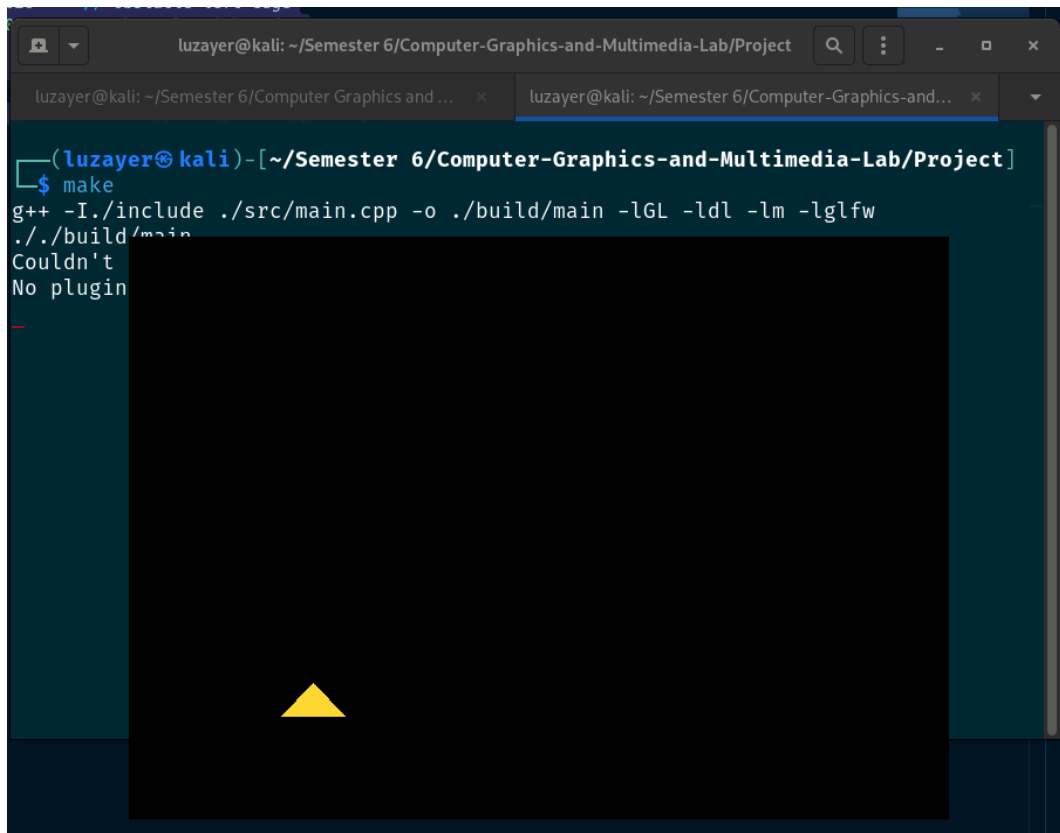
    ox0 = obstacle.x - 0.028 // obstacle left boundary
    ox1 = obstacle.x + 0.028 // obstacle right boundary
    oy0 = GROUND_Y           // obstacle lower edge
    oy1 = GROUND_Y + 0.30    // obstacle upper edge

    return ( px1 > ox0 AND px0 < ox1 , AND py1 > oy0 AND py0 < oy1 )

```

5. Terminal Screenshots with Output

5.1 Build Output

A terminal window screenshot from a Kali Linux machine. The window title is 'luzayer@kali: ~/Semester 6/Computer-Graphics-and-Multimedia-Lab/Project'. The prompt is '(luzayer@kali)-[~/Semester 6/Computer-Graphics-and-Multimedia-Lab/Project]'. The user has entered '\$ make'. The output shows the compiler command: 'g++ -I./include ./src/main.cpp -o ./build/main -lGL -ldl -lm -lglfw'. Below this, it says '././build/main' and then 'Couldn't' and 'No plugin'. A large black rectangular area obscures the rest of the terminal output. A yellow triangle cursor is visible at the bottom of the blacked-out area.

```
(luzayer@kali)-[~/Semester 6/Computer-Graphics-and-Multimedia-Lab/Project]
$ make
g++ -I./include ./src/main.cpp -o ./build/main -lGL -ldl -lm -lglfw
././build/main
Couldn't
No plugin
```

5.2 Project File Tree

After cleanup, the project is reduced to its minimal form:

```
Project/
├── Makefile
├── build/
│   └── main
├── include/
│   ├── glad.h
│   └── glfw3.h
└── src/
    ├── glad.c
    └── main.cpp
```

5.3 Window Title -Live Score Display

State	Window Title
Start	Triangle Jump
Score 3	Score: 3
Score 10	Score: 10
Game Over	SPACE to restart

5.4 Game Visuals Description

State	What Appears
Playing	Yellow triangle on a black background, cyan rectangles come in from the right
Jumping	The yellow triangle arcs upward with a kind of gravity curve, pretty “realistic” though not exactly measured
Game Over	Then the triangle turns red, and all obstacles freeze visually, like the scene got paused
Restart	When you press SPACE the screen instantly snaps back to the initial start state, no in between

6. Discussion and Future Work

6.1 Discussion

The project shows the core OpenGL ideas in a working, interactive setup :

- **Shader Uniforms as a Game State bridge:** Instead of sending vertex data again each frame, the game uses two uniforms -offset (vec2) and color (vec4) -to nudge and recolor the objects. That’s more efficient, and it’s idiomatic for small 2D games
- **Single compilation Unit:** By including glad.c right into main.cpp, it builds as one translation unit, which keeps the build step basically one line, but the code stays all in view in the same file
- **Euler Integration:** The physics uses a semi implicit Euler loop (velY += accel * dt, posY += velY * dt). It’s pretty standard for this kind of game tick, simple, steady, and accurate enough for the scale here

- **AABB Collision:** Axis-Aligned Bounding Box collision is used because it's light and fast. The hitboxes are purposely slightly inset, smaller than the visible shape, so it feels fair to the player
- **Difficulty curve:** The ramp feels exponential-ish, with speed going up and the spawn interval shrinking every 5 points. That creates a good challenge progression you often see in arcade style games

6.2 Limitations

- **No text rendering:** OpenGL doesn't include a built in text rendering option, so the score gets sent through the window title bar instead, kind of a workaround more than a real in game HUD or overlay.
- **No audio:** There aren't any sound effects or background music, it's currently silent, like fully silent, no ambient audio either.
- **Fixed obstacle height:** Every obstacle is the same size, so yeah, variable height obstacles would add more variety to the gameplay.
- **No high score persistence:** The score just disappears once you close the window, so you lose your best run every time.

6.3 Future Work

Enhancement	Description
In-game HUD	Render the score using a bitmap font (example: stb_truetype) right inside the OpenGL viewport
Variable obstacles	Randomly change obstacle width AND height to make it a bit harder
Double jump	Permit one extra jump while airborne for more skilled play
Background parallax	Scroll multiple background layers to create that speed vibe
Audio	Add jump + collision sound cues using OpenAL or miniaudio
High score	Save the best score to a small local file so it sticks around
Mobile-style tap input	Port the whole thing to OpenGL ES and support touch input

7. Conclusion

Triangle Jump is a complete, working 2D arcade game, it's written in about 200 lines of C++ and uses raw OpenGL 3.3 Core Profile plus GLFW, with basically zero external game engine dependencies . The project shows:

- **Window creation and OpenGL context setup** using GLFW
- **GPU geometry upload** via VAOs and VBOs
- **GLSL shader programming** for vertex transformation and fragment coloring
- **Real-time game loop** with delta-time physics
- **Procedural obstacle spawning** with adaptive difficulty
- **AABB** collision detection
- **Single-file build system** using make

The game is intentionally spare on the visual side - you only see shapes, geometric primitives over a plain black background -but it's mechanically complete as an endless runner. That mix of simplicity with real functionality makes it a good little lab exercise, to understand the OpenGL rendering pipeline in a direct, hands-on, interactive way.